

AWS Practice — A Step-by-Step Guide for Beginners

From zero to hands-on: set up, understand the core ideas, and build real services safely.

This guide assumes you're new to AWS. It walks you from creating an account through building real services (storage, compute, a database, and a serverless function), one careful step at a time. You'll use two ways to work with AWS — the web Console (to see things) and Terraform code (to build things repeatably). Follow it in order; each section builds on the last. Nothing here should cost money if you follow the safety rules.

1. What Is AWS, and What Are You Learning?

- **Cloud in one line:** AWS (Amazon Web Services) is a cloud provider: instead of buying your own servers, you rent computing resources (storage, servers, databases) over the internet and pay only for what you use.
- **What's a 'service'?** A 'service' is one building block AWS offers — S3 for file storage, EC2 for virtual servers, DynamoDB for a database, Lambda for running code, IAM for permissions. You combine services to build systems.
- **Your goal:** You will practice five core services (S3, IAM, EC2, DynamoDB, Lambda) — enough to understand how real cloud systems and data pipelines are built.
- **Three ways to work:** Three ways to use AWS: the Console (a website you click), the CLI (typing 'aws' commands in a terminal), and Infrastructure as Code / Terraform (writing files that build resources for you). You'll use all three — mostly Terraform, because it's repeatable and what professionals use.

2. One-Time Setup (Do This First)

Complete these before building anything. It takes about 30-45 minutes.

2.1 Create an AWS Account

1. Go to aws.amazon.com and click 'Create an AWS Account'. You'll need an email, a password, and a credit/debit card (AWS requires one even for the free tier; you won't be charged if you stay within limits).
2. Choose the 'Basic' (free) support plan when asked.

2.2 Secure the Root User (critical)

Important: The email you signed up with is the 'root user' — it has unlimited power over the account. Protect it and don't use it for daily work.

3. Turn on MFA (Multi-Factor Authentication) for the root user: sign in, go to the account menu -> Security Credentials -> enable MFA using an app like Google Authenticator or Authy on your phone.
4. Create an admin IAM user for everyday use (next section) instead of using root.

2.3 Create an Admin IAM User (for daily use)

5. In the Console, search for 'IAM' -> Users -> Create user. Give it a name like 'admin'.
6. Attach the 'AdministratorAccess' policy (for learning; real jobs use tighter permissions).
7. Enable console access and set a password if you want to log in as this user in the browser.
8. Create an access key for this user: IAM -> Users -> your user -> Security credentials -> Create access key -> choose 'Command Line Interface (CLI)'. Copy the Access Key ID and Secret Access Key — the secret is shown ONCE.

Important: Never share access keys, never paste them into chats or emails, and never commit them to GitHub. Treat the secret like a password.

2.4 Install the Tools

- AWS CLI — the command-line tool to talk to AWS.
- Terraform — the Infrastructure-as-Code tool you'll build with.
- Git + a GitHub account — to save your work.
- A code editor — VS Code or PyCharm.

```
# macOS (using Homebrew):  
brew install awscli terraform git  
  
# verify each is installed:  
aws --version  
terraform -version  
git --version
```

2.5 Connect the CLI to Your Account

```
aws configure  
#   AWS Access Key ID:      <paste your key>  
#   AWS Secret Access Key: <paste your secret>  
#   Default region name:   us-west-2  
#   Default output format: json  
  
# confirm it works (should print your account/user):  
aws sts get-caller-identity
```

Tip: A 'region' (like us-west-2, Oregon) is a geographic location where your resources live. Pick one and stick with it while learning — us-west-2 is a good default.

3. Core Concepts Every Beginner Needs

- **Regions & AZs:** The world is divided into Regions (e.g., us-west-2 = Oregon), each containing multiple Availability Zones (isolated datacenters). Keep all your practice in one region.
- **Free Tier & cost:** You pay per use, but AWS has a Free Tier (12 months for many services, plus some always-free). Staying inside it keeps practice free — but you **MUST** clean up resources when done.
- **IAM:** Identity and Access Management controls who can do what. Users (people), groups (collections of users), roles (assumed by services), and policies (permission documents). Everything in AWS is permission-gated by IAM.
- **Infrastructure as Code:** Instead of clicking in the Console, you write Terraform files describing what you want, and Terraform builds it. It's repeatable, reviewable, and undo-able (destroy). This is the professional way and what you'll practice most.

4. The Terraform Workflow (Your Daily Loop)

Every time you build with Terraform, you run the same handful of commands. Learn this loop by heart.

```
terraform init      # first time / after adding a provider: downloads plugins
terraform fmt       # auto-formats your code neatly
terraform validate  # checks for errors BEFORE touching AWS
terraform plan      # PREVIEW: shows exactly what will be created/changed
terraform apply     # builds it for real (you type 'yes' to confirm)
terraform output    # shows results (IPs, names, ARNs)
terraform destroy   # deletes everything it created (stops billing)
```

Important: Always read 'terraform plan' before 'apply'. It's your one chance to catch a mistake before it touches your live account. And always 'terraform destroy' when you finish practicing.

5. Learn By Building (The Ordered Path)

Build these services in this order — each is a bit more involved than the last. For each, you create a small Terraform folder, run the workflow above, verify it worked, then destroy it. (A ready-made modular project with all of these already exists in your GitHub as 'terraform-aws-practice' — you can study it as a reference.)

Service 1 — S3 (Storage): start here, it's the simplest

- **What:** Object storage — a place to put files ('objects') in 'buckets'. The foundation of almost every data system.

Real-world example: When you upload a photo to Instagram or a document to Dropbox, the file is stored in object storage exactly like S3. Netflix stores its movie files in S3; a data team stores millions of raw sales records in S3 as the 'data lake' that reports are later built from. Think of S3 as an infinitely large, super-durable Google Drive that applications (not just people) read and write.

- **Build:** Create a bucket with versioning + encryption + public access blocked.
- **Verify:** `aws s3 ls` (should list your bucket) and upload a test file in the Console.
- **Beginner note:** Bucket names are globally unique across all AWS accounts — if a name is taken, add something unique like your account ID.

Service 2 — IAM (Permissions): understand the security model

- **Build:** Practice creating a group, a user in that group, and a policy — the pieces that control access.

Real-world example: Think of a company office building. IAM is the security desk that issues badges. A 'user' is an employee's badge; a 'group' is a department (all Finance badges open the same doors); a 'policy' is the rule sheet saying which doors a badge opens; a 'role' is a temporary visitor pass given to a contractor for one day. In AWS, a delivery-app's payment service gets a 'role' that only opens the 'charge card' door — nothing else — so if it's ever compromised, the damage is contained.

- **Why now:** This teaches you the model that gates every other service. Take your time here.

Service 3 — EC2 (Compute): your first server

- **Build:** A virtual machine you can run software on. Launch a Free-Tier t3.micro with a web server.

Real-world example: EC2 is a rented computer in the cloud. When a small business runs its website, or a startup hosts the backend of its mobile app, it often runs on EC2 servers instead of a computer in their office. It's like renting a PC by the hour that's always on, connected to fast internet, and reachable from anywhere — you install your software on it and it serves your users. Bigger AWS services like the EMR data-processing cluster are really just fleets of EC2 machines working together.

- **Verify:** Open its public IP in a browser and see your test web page.
- **Beginner note:** A 'security group' is a firewall controlling what traffic can reach it — you open SSH (22) and HTTP (80).

Service 4 — DynamoDB (Database): store structured data

- **Build:** A NoSQL database. Create an 'Orders' table with a partition key (order_id) and sort key (order_date).

Real-world example: When you order food on a delivery app and watch the status change from 'Preparing' to 'Out for delivery,' a database like DynamoDB is holding that live order record and updating it in milliseconds. It's built for apps with millions of users where every read/write must be instant — a shopping cart, a game leaderboard, a user-profile store. Your 'Orders' table is a miniature version of exactly that.

- **Verify:** `aws dynamodb describe-table --table-name Orders`, then add an item in the Console.
- **Beginner note:** You design the table around how you'll query it, up front — different from a SQL database.

Service 5 — Lambda (Serverless): run code without a server

- **Build:** A function that runs on demand. Build one that writes an order into your DynamoDB table.

Real-world example: Imagine every time a customer places an order, a small piece of code needs to run to save it, send a confirmation email, and notify the restaurant. Lambda runs that code automatically when the order arrives — with no server sitting idle waiting, and you pay only for the split-second it runs. Real examples: resizing a photo the moment it's uploaded to S3, processing a payment when a button is clicked, or (like yours) writing an order into DynamoDB. It's the 'glue' that reacts to events and connects other services together.

- **Verify:** Invoke it with a test event and confirm the item appears in DynamoDB.
- **Beginner note:** This ties services together — the Lambda needs an IAM role to get permission to write to DynamoDB. That's the whole security model coming together.

6. Safety & Cost Rules (Read Twice)

- **Free Tier:** You have a limited Free Tier. It covers small usage of most services for 12 months. Small practice stays free.
- **Set a billing alarm:** Console -> Billing -> Budgets -> create a budget (e.g., alert me if I spend over \$5). This emails you before any surprise.
- **ALWAYS destroy:** When you finish a practice session, run 'terraform destroy'. A running EC2 instance or forgotten resource is the #1 cause of unexpected charges.
- **Never commit secrets:** Never commit access keys, secrets, or Terraform state files to GitHub. Use a .gitignore (your practice repos already have one).
- **Guard root:** Protect the root user with MFA and use your admin IAM user for daily work.

7. Common Beginner Mistakes (and Fixes)

- **Auth errors:** 'InvalidClientTokenId' / 403 -> your credentials are wrong or expired. Fix: re-run 'aws configure' with a valid key; check 'aws sts get-caller-identity'.
- **Name collisions:** 'BucketAlreadyExists' -> the bucket name is taken globally. Fix: choose a more unique name (e.g., add your account ID).
- **Free-Tier type:** 'not eligible for Free Tier' -> you picked the wrong instance type. Fix: use t3.micro, or run 'aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true'.
- **Init errors:** Forgetting to 'terraform init' after adding a new provider -> Fix: just run 'terraform init' again.
- **Charges:** Surprise bill -> you left something running. Fix: 'terraform destroy', and check the Billing dashboard for active resources.

Tip: When you hit an error, read it carefully — it usually tells you exactly what's wrong. Then fix that one thing and re-run 'terraform plan'.

8. A 4-Week Beginner Practice Plan

Week 1 — Foundations

- Create and secure your account (MFA, admin user). Install tools. Configure the CLI. Read Sections 1-4 of this guide.
- Build and destroy your first S3 bucket with Terraform. Upload a file in the Console.

Week 2 — Permissions & Compute

- Practice IAM: create a group, user, and policy. Understand users vs roles.
- Launch a Free-Tier EC2 instance, view its web page, then destroy it.

Week 3 — Data

- Create a DynamoDB table, add and query items.
- Learn the difference between a database (DynamoDB) and storage (S3).

Week 4 — Putting It Together

- Build a Lambda that writes to DynamoDB, with its IAM role. Invoke it and verify.
- Combine everything in one modular Terraform project (like your 'terraform-aws-practice' repo). Practice 'plan/apply/destroy' until it feels natural.

9. Quick Glossary

- **AWS:** Amazon Web Services — the cloud provider.

- **Region:** a geographic location of datacenters (us-west-2 = Oregon).
- **Console:** the web interface where you click to manage AWS.
- **CLI:** Command Line Interface — the 'aws' terminal tool.
- **IaC:** Infrastructure as Code — building resources from code (Terraform).
- **IAM:** Identity and Access Management — the permission system.
- **S3:** object storage for files.
- **EC2:** virtual servers.
- **DynamoDB:** a NoSQL database.
- **Lambda:** run code without managing servers.
- **Free Tier:** the usage AWS gives you for free (mostly for 12 months).

Where to Go Next

- Study your own 'terraform-aws-practice' GitHub repo — it has all five services as reusable modules.
- When comfortable, learn data-engineering services: Glue (ETL), Athena (query S3), Redshift (warehouse), Kinesis (streaming).
- Pair this with your other prep docs (the AWS Data Engineer 30-Day plan and the Terraform Playbook) in this same folder.